2027-I

UNAM

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN



MATERIA

Web 2

TEMA

Request_Response_Json

PROFESOR

Mtro. ISC. MIGUEL ANGEL SÁNCHEZ HERNÁNDEZ

Tabla de contenido

1. Inicio 3
2. Instalar bcryptjs 3
3. Modificar index.js 3
4. Modificar inicio_router.js 3
5. Modificar inicioController.js 4
6. Correr la aplicación 6

1. Inicio

Abrir el proyecto base que se creo en la práctica uno

2. Instalar bcryptjs

Instalar bcryptjs

```
npm i bcryptjs
```

3. Modificar index.js

```
import express from "express";
import inicio from "./routes/inicio_router.js";

//Crear la aplicación
const app = express();

//accesos a los datos del formulario
app.use(express.urlencoded({extended:true}))

// Middleware para parsear JSON
app.use(express.json());

//carpeta publica
app.use(express.static("public"));

//routing
app.use("/", inicio);

//definiendo el puerto
const port = 2800;
app.listen(port, () => {
  console.log(`Esperando peticiones en el puerto ${port}`);
});
```

4. Modificar inicio_router.js

```
import express from "express";
import {inicio,registro,usuarios_lista,login} from "../controllers/inicioController.js"

const router=express.Router();
//Routing
router.get('/',inicio);
router.post("/registro",registro)
router.get('/lista',usuarios_lista);
router.post("/login",login)

export default router
```

5. Modificar inicioController.js

```
import express from "express";
import bcrypt from "bcryptjs";

// "Base de datos" en memoria
const usuarios = [];

const inicio = (req, res) => {
  res.render("index.html");
};

const registro = async (req, res) => {
  try {
    const { nombre, email, password } = req.body;

    // 1. Validar campos obligatorios
    if (!nombre || !email || !password) {
      return res.status(400).json({
        mensaje:
          "clearTodos los campos son obligatorios (nombre, email, password)",
      });
    }

    // 2. Validar formato de email (básico)
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
      return res.status(400).json({ mensaje: "Email inválido" });
    }

    // 3. Validar longitud de contraseña
    if (password.length < 6) {
      return res.status(400).json({
        mensaje: "La contraseña debe tener al menos 6 caracteres",
      });
    }

    // 4. Verificar si el email ya está registrado
    const existe = usuarios.find((u) => u.email === email);
    if (existe) {
      return res.status(409).json({ mensaje: "El email ya está registrado" });
    }

    // 5. Encriptar la contraseña
    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(password, salt);

    // 6. Crear el usuario
    const nuevoUsuario = {
      id: usuarios.length + 1,
      nombre,
      email,
```

```
      password: passwordHash,
      fechaRegistro: new Date().toISOString(),
    };

    usuarios.push(nuevoUsuario);

    // 7. Responder (sin devolver la contraseña)
    res.status(201).json({
      mensaje: "Usuario registrado exitosamente",
      usuario: {
        id: nuevoUsuario.id,
        nombre: nuevoUsuario.nombre,
        email: nuevoUsuario.email,
        fechaRegistro: nuevoUsuario.fechaRegistro,
      },
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ mensaje: "Error interno del servidor" });
  }
};

const usuarios_lista = async (req, res) => {
  const lista = usuarios.map((u) => ({
    id: u.id,
    nombre: u.nombre,
    email: u.email,
    fechaRegistro: u.fechaRegistro,
  }));
  res.json(lista);
};

const login = async (req, res) => {
  const { email, password } = req.body;

  const usuario = usuarios.find((u) => u.email === email);
  if (!usuario) {
    return res.status(401).json({ mensaje: "Credenciales inválidas" });
  }

  const passwordValida = await bcrypt.compare(password, usuario.password);
  if (!passwordValida) {
    return res.status(401).json({ mensaje: "Credenciales inválidas" });
  }

  res.json({
    mensaje: "Login exitoso",
    usuario: { id: usuario.id, nombre: usuario.nombre, email: usuario.email },
  });
};

export { inicio, registro, usuarios_lista,login };
```

6. Correr la aplicación

En la terminal correar la aplicación

```
npm run server
```